Project: Voice Controlled Task Manager

Build a web application where users can manage tasks completely through voice interaction.

The assistant should allow users to:

- Create tasks
- Read tasks
- Update tasks
- Delete tasks

Everything should happen through voice conversation.

There should be:

- No typing for task actions
- No edit buttons
- No delete buttons
- No manual CRUD interactions

The assistant should:

- Listen to the user
- Respond back using voice
- Handle real-time conversations naturally

The experience should feel like a real AI voice agent instead of a normal chatbot.

The project should include:

- Speech-to-Text (STT)
- Text-to-Speech (TTS)
- Real-time voice interaction
- Voice-based CRUD operations
- Conversational AI workflows
- Context-aware responses

Storage can be handled using:

- Database (MySQL,PostgreSQL, etc.)
- Local storage
- Google Spreadsheet

Authentication is optional but good to have:

- Signup
- Login
- Session handling

Example Flows

Create a Task

User:
“Create a task for syncing with the product manager at 10 AM.”

Expected:

- Assistant understands the request
- Task gets created
- Assistant confirms the action using voice

User:
“Create a task for posting on LinkedIn at 5 PM.”

Expected:

- Task gets added successfully
- Assistant responds naturally

Update a Task

User:
“Change the LinkedIn task to 6 PM.”

Expected:

- Assistant identifies the correct task
- Assistant updates the task
- Assistant confirms the change

Context Handling

User:
“Actually change the previous one to 7 PM.”

Expected:

- Assistant understands which task the user is referring to
- Conversation context should be maintained naturally

Delete a Task

User: “Delete the 9:15 task.”

If the request is unclear, the assistant should ask follow-up questions.

Example:

Assistant: “I could not find a 9:30 task. Did you mean the 9:15 task?”

User: “Yes.”

Expected:

- Assistant confirms before deleting
- Assistant deletes the correct task only after confirmation

Read Tasks / Agenda Understanding

User: “What are today’s evening tasks?”

or

“Give me a brief about today’s agenda.”

Expected:

- Assistant should understand time-based context naturally such as:
    - today
    - tomorrow
    - evening
    - morning
    - afternoon
- Assistant should summarize tasks conversationally instead of only listing them
- Assistant should maintain context during follow-up questions

Example:

Assistant: “You have a product sync at 6 PM and a LinkedIn post scheduled at 8 PM.”

User: “Move the second one to tomorrow.”

Expected:

- Assistant should understand which task the user is referring to
- Context should be maintained naturally throughout the conversation

Semantic Understanding

User: “Move my evening workout.”

Expected:

- Assistant should understand the meaning naturally instead of relying only on exact task names

Interruption Handling

While the assistant is speaking:

Assistant: “I found three matching tasks”

User interrupts: “No, delete the LinkedIn one.”

Expected:

- Assistant should stop playback immediately
- Assistant should continue the conversation naturally
- Real-time interruption handling should work smoothly

Multiple Task Handling

User: “Create three tasks for tomorrow morning. Gym at 7 AM, team sync at 9 AM, and post on LinkedIn at 11 AM.”

Expected:

- Assistant should process multiple requests efficiently
- Tasks should be created correctly
- Assistant should respond naturally

Failure Handling

The system should gracefully handle situations such as:

- unclear commands
- STT failures
- TTS failures

- WebSocket disconnects
- LLM timeout issues

The conversation should recover naturally whenever possible.

Reliability Expectations

The assistant should:

- validate actions before execution
- confirm destructive actions before deleting
- handle retries or fallback responses for invalid operations
- maintain smooth conversational flow with low response latency

Candidates may structure the application using multiple agents if needed, such as:

- Planner agent
- Conversation agent
- Task execution agent
- Scheduling agent

Deployment & Submission

Please submit:

- GitHub repository
- Setup instructions
- Live demo link if available

Candidates can deploy the project using any free hosting platform or free domain service. A production-grade deployment is not required.

Platforms such as:

- GitHub Pages
- Vercel
- Netlify
- Render
- Any other free hosting service is completely acceptable.

A working live demo link is preferred so the project can be tested easily.

Estimated completion time:
1–2 days